ABV – Indian Institute of Information Technology & Management
Gwalior, Madhya Pradesh

DEVELOPMENT OF PSEUDO RANDOM PATTERN GENERATOR USING CPLD FOR TESTING OF LOW VOLTAGE DIFFERENTIAL SIGNALLING [LVDS] TRANSMITTER AND RECEIVER SYSTEM

A Project Report of Two Months Industrial Training Submitted in Partial Fulfilment of The Requirement for the Award of The Degree of

B. Tech. in Electrical and Electronics Engineering

(May-July, 2026)

Submitted By:

Kiran Meena

ST00910

In

VTAD (VLSI TEST AND APPLICATION DIVISION)
SEMICONDUCTOR LABORATORY, MOHALI
PUNJAB

1. BONAFIDE CERTIFICATE

This is to certify that the project report entitled "**Development of Pseudo Random Pattern Generator using CPLD for Testing of Low Voltage Differential Signalling (LVDS) Transmitter and Receiver System**" submitted to ABV – Indian Institute of Information Technology & Management, Gwalior is a Bonafide record of work done by **Kiran Meena** under my supervision from 18th May, 2026 to 14th July, 2026 at the VLSI Testing Division (VTAD), Semiconductor Laboratory, Mohali.

Jagjeet Singh

Head of Department
VTAD/Semiconductor Laboratory
MeitY, Government of India

Gaurav Kumar Srivastava

Scientist/Engineer 'SF'
VTAD/Semiconductor Laboratory
MeitY, Government of India

Place: Mohali
Date: 14-07-2026

2. ABSTRACT

Testing a digital system reliably requires input data that exercises every possible bit transition — not a fixed, predictable pattern, but something that closely resembles real-world data. Random patterns serve this purpose effectively. In this project, developed a **Pseudo Random Pattern Generator** using a **CPLD (Altera MAX EPM7256AE)** and used it to test and characterise a **Low Voltage Differential Signalling (LVDS)** transmitter and receiver pair across different data rates.

Before implementation, studied pseudo-random pattern generation techniques — particularly the **Linear Feedback Shift Register (LFSR)** method, which was used in this project to generate a 10-bit maximal-length pseudo-random sequence. Also studied the internal architecture of the MAX 7000AE CPLD — its Logic Array Blocks, macrocells, Programmable Interconnect Array, and In-System Programmability — and the fundamentals of LVDS differential signalling.

Software simulation performed using **ModelSim-Altera 10.1d** with a 24 MHz crystal clock-based VHDL code. Simulated each module independently — the clock generation circuit, the clock divider producing 24 MHz, 12 MHz, 6 MHz, and 3 MHz outputs, the 10-bit LFSR pseudo-random generator, and the comparator with error detection logic — before verifying the complete integrated design. All modules produced correct waveforms.

For hardware development, **Quartus II 13.1** with direct external clock-based VHDL code. Synthesized the design, assigned physical pins on the 208-pin CPLD using the Pin Planner, and successfully downloaded the compiled code onto the device. The design utilized only 26 macrocells and 19 active pins — well within the device's capacity.

The hardware was tested using an oscilloscope. The LVDS pair operated **error-free from 1.2 MHz to 40 MHz**. Signal integrity degradation was observed at 45 MHz, and clear bit errors were detected at 50 MHz, establishing the reliable upper operating limit of the system. Propagation delay through the LVDS link remained consistent at approximately 4.0 to 4.4 ns across all test frequencies, confirming a stable differential link.

3. ACKNOWLEDGEMENT

The completion of this industrial training and the preparation of this report would not have been possible without the support, guidance, and encouragement of several individuals to whom I owe my sincere gratitude.

I would like to begin by expressing my deepest gratitude to **Mr. Jagjeet Singh**, Head of VTAD, Semiconductor Laboratory, Mohali, for granting me the opportunity to undertake this training at such a prestigious research institution. His leadership, cordial support, and logistical guidance provided the right environment for me to learn and grow throughout this project.

I am especially thankful to my project mentor, **Mr. Gaurav Kumar Srivastava**, Scientist/Engineer 'SF', VTAD, whose patient guidance and technical expertise shaped every stage of this work. His ability to explain complex concepts clearly, his constant encouragement during challenging phases, and his dedication to helping me understand the project deeply rather than just completing it — all of this made an immense difference to my learning experience.

Finally, I am grateful to my institution, ABV-IIITM Gwalior, for supporting industrial training as an integral part of the curriculum. This experience has not only strengthened my technical foundation but has also given me a clearer sense of direction for my professional journey ahead.

TABLE OF CONTENTS

LIST OF FIGURES

4. INTRODUCTION TO THE ORGANIZATION

4.1 About Semi-Conductor Laboratory (SCL)

Figure 1: Semiconductor Laboratory

Semi-Conductor Laboratory (SCL) is an autonomous body under the Ministry of Electronics and Information Technology (MeitY), Government of India. It is the only facility in the country that handles the complete process of chip development in-house, covering design, fabrication, assembly, packaging, testing, and reliability assurance of ASICs, opto-electronic devices, and MEMS devices.

SCL was earlier known as Semiconductor Complex Limited and was converted into a Semi-Conductor Laboratory under the Department of Space in 2006. Administrative control of SCL was later moved to MeitY in 2022.

SCL operates an 8-inch wafer fabrication line based on 180 nm CMOS technology, along with a 6-inch line used for MEMS development. Devices made at SCL are used in applications that demand very high reliability, including space systems, where field failures cannot easily be corrected later. This is why testing at SCL is given as much importance as the design process itself.

4.2 Relevance to This Project

SCL designs devices such as LVDS transmitters and receivers, which are used for high-speed data transfer in onboard electronics. A chip design is only as reliable as the testing carried out on it, and this is the area this project relates to. Instead of testing an LVDS pair with manually generated or fixed signals, a pseudo random pattern generator is used, since it gives a controlled and repeatable signal that still behaves like real, unpredictable data for testing purposes. This project, though done at a CPLD prototyping level, follows the same basic testing approach used at SCL.

5. UNDERSTANDING PSEUDO-RANDOM PATTERNS

5.1 Why Pseudo-Random Patterns Are Needed for Testing

Before getting into how pseudo-random patterns are generated, it helps to understand why they are needed in the first place. When testing a digital system such as an LVDS transmitter-receiver pair, the input data used for testing matters a lot. A fixed or simple pattern, such as a repeating sequence of 1s and 0s, only exercises a limited set of signal transitions and does not represent how real data actually behaves.

Real data changes unpredictably, both in value and in timing, and it is exactly this unpredictability that puts stress on a system's timing margins, noise immunity, and overall signal integrity. A test pattern is only useful if it can recreate this kind of unpredictable behaviour. At the same time, the test pattern also needs to be known in advance, so that the output expected from the system under test can be predicted and checked against the actual output. A truly random source cannot do this, since there is no way to know beforehand what value is coming next.

This is exactly the gap that pseudo-random patterns fill — they behave like random data for the purpose of testing, while still being generated by a known, repeatable process. This combination is what makes them the standard choice for testing high-speed digital interfaces such as the LVDS pair used in this project.

5.2 Random Pattern Generators

A random pattern generator is basically a system that produces a sequence of values with no fixed pattern — values that look irregular and unpredictable. This isn't a new idea at all; long before computers, things like coin tosses, dice, and card shuffling were already being used as random pattern generators, just manual ones. The problem with these older methods is scale — generating a long sequence of random values this way needs a lot of manual effort and time. Once computers came into the picture, generating random-looking sequences became far quicker, and today this same idea is used across simulation, gaming, lottery systems, and — relevant to this project — digital testing.

5.3 Pseudo-Random Generator

The pattern used in this project is not actually random in the true sense; it is what is called pseudo-random. A pseudo-random number generator (PRNG) produces a sequence that looks random but is generated by a fixed algorithm, starting from an initial value called the

seed. Because the algorithm and seed are both fixed, the entire output sequence is fully determined in advance — running the same generator with the same seed will always give the exact same sequence.

This is the part that initially seemed contradictory while learning about it — how can something be called "random" if it is completely predictable? The reasoning becomes clear once it is connected back to the actual testing requirement explained above: a generator that is reproducible is exactly what is needed for testing, since the correct output expected at the receiving end is always known beforehand. If the generated pattern were truly random, there would be nothing fixed to compare the received signal against.

5.4 Linear Feedback Shift Register (LFSR)

The method used in this project to generate the pseudo-random sequence is the Linear Feedback Shift Register, or LFSR. An LFSR is a type of shift register where the input bit fed into the register is not an outside signal, but is generated from the register's own previous state, using an XOR operation on selected bits within the register, referred to as tap positions. On every clock cycle, all bits shift one position forward, and the XOR feedback produces the new bit entering the register.

Since the register has only a limited number of possible states, the sequence it produces has to repeat eventually — there is no way around that with a finite number of bits. What can be controlled, though, is how long the sequence runs before it repeats, and this depends entirely on which tap positions are chosen for the feedback. A well-chosen set of taps gives what is called a maximal-length sequence, which behaves like a random sequence for a very long stretch before it cycles back.

The structure of an LFSR can be represented using a chain of D flip-flops, where the output of one feeds into the next, and the chosen tap outputs are combined through XOR gates to generate the feedback bit fed back into the first flip-flop. This same operation can also be written as a state-transition matrix, where the next state of the register, X(t+1), is calculated from its present state, X(t), using the shift-and-feedback relationship.

For this project, a 10-bit LFSR was used to generate the pseudo random sequence, and 4 bits were tapped out from it and sent to the LVDS transmitter as the test data.

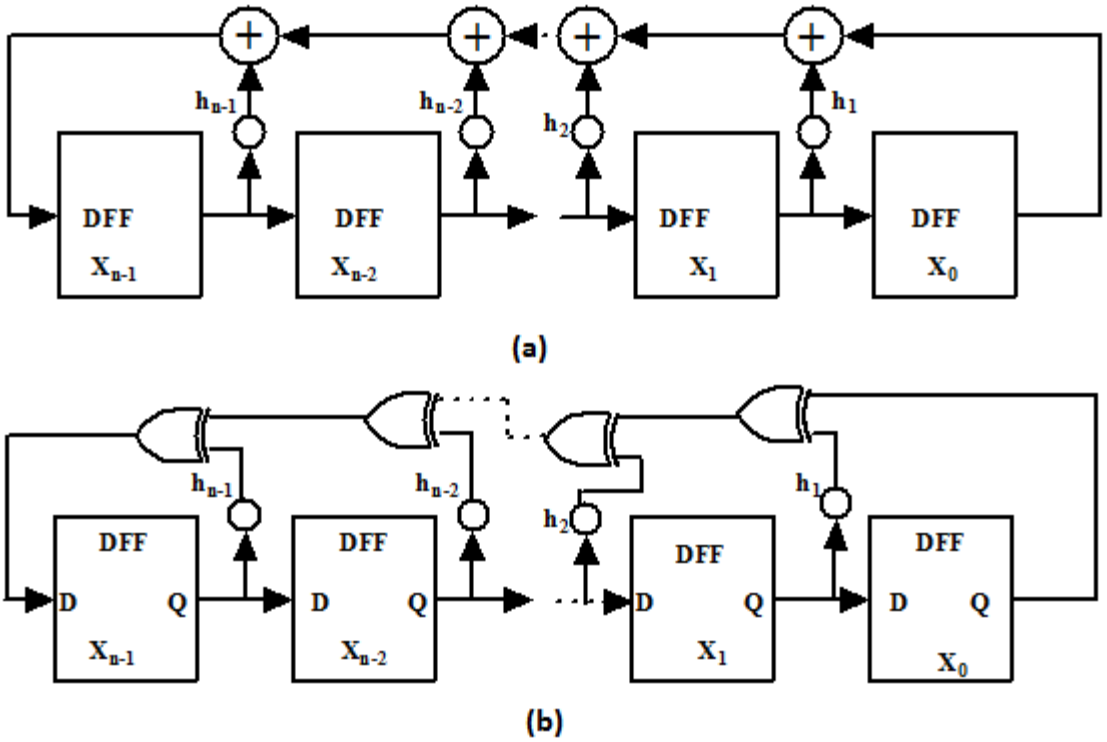


Figure 2: Standard LFSR

6. HARDWARE AND SOFTWARE SELECTION

6.1 Software Tools Used

Two software tools were used in this project, each serving a different stage of the design process.

ModelSim (Altera Edition, 10.1d) is a simulation and verification tool that supports VHDL, Verilog, SystemVerilog, and mixed-language designs. It allows a VHDL design to be run against defined test conditions purely in software, with the resulting waveforms observed to verify logic correctness — without involving any hardware at all. Simulation was carried out first in this project, since verifying logic in software is faster than debugging on actual hardware, and avoids the risk of mistaking a hardware fault for a logic fault.

Quartus II is the development and synthesis tool used for hardware implementation. It takes the verified VHDL design, compiles it into a form the target device can run, allows assignment of design signals to physical pins on the CPLD package, and finally programs the compiled design onto the device via JTAG. This is the stage where the design moves from being just code to becoming an actual working circuit running on the Altera MAX EPM7256AE CPLD.

This two-tool flow makes clear why simulation and synthesis are treated as separate stages — simulation confirms whether the logic itself is correct, while synthesis and hardware testing confirm whether that logic behaves correctly once running on real hardware, with real timing constraints, real noise, and real signal delays that simulation does not fully capture.

6.2 Hardware Selection — Altera MAX EPM7256AE

The device used in this project is the **Altera MAX EPM7256AEQC208-7** from the MAX 7000AE family. Its key specifications are below:

Parameter	Value
Device family	MAX7000AE
Macrocells	256
Package	PQFP, 208 pins
Usable I/O pins	164
Speed grade	-7
Supply voltage	3.0 V – 3.6 V
Operating temperature	0 °C to 85 °C

The design used only 26 of the 256 available macrocells and 19 of the 164 usable I/O pins — confirming the device was well-sized for this application with significant headroom remaining.

6.2.1 Why a CPLD and Not an FPGA

For a design of this scale — a clock divider, a shift-register-based pattern generator, and a bit comparator — the logic required is relatively limited. It does not justify the larger reconfigurable fabric, internal memory blocks, or DSP resources offered by an FPGA. A CPLD is better suited to designs of this size and complexity.

More importantly, CPLD logic is implemented using non-volatile EEPROM cells, unlike the volatile SRAM-based configuration typically used in FPGAs. This means the design stays programmed even when power is removed — the device is ready to operate immediately at power-on, without waiting for configuration to reload from external memory. For a test fixture like this one, which is expected to retain its configuration across power cycles without reloading, this is a significant practical advantage.

6.2.2 Internal Architecture of the CPLD

The MAX 7000AE architecture is built around five elements. Understanding these was necessary before synthesis — because the tool maps VHDL constructs directly onto these physical structures.

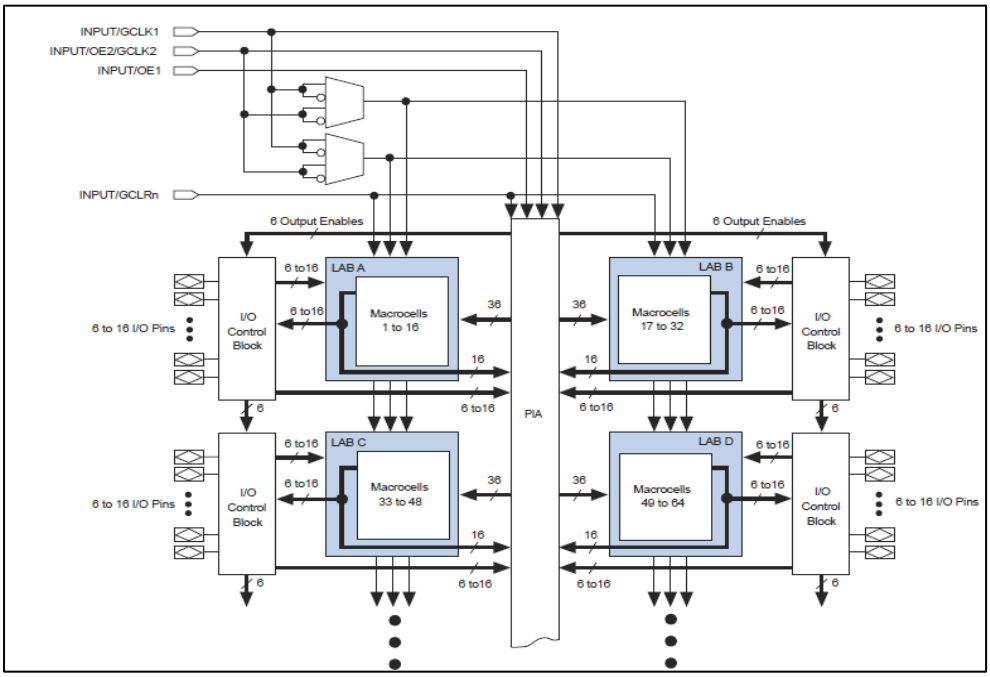


Figure 3: MAX 7000 AE CPLD

- **Logic Array Blocks (LABs):** The device architecture is based on an array of high-performance LABs, each consisting of 16 macrocells. Each LAB is fed by 36 signals from the PIA for general logic inputs, global control signals for secondary register functions, and direct input paths from I/O pins for fast setup times. Multiple LABs are linked together through the PIA — a global bus fed by all dedicated input pins, I/O pins, and macrocells.
- **Macrocells:** The macrocell is the basic logic unit within a LAB and can be configured for either combinatorial or sequential logic operation. Each macrocell consists of three parts — the logic array, the product-term select matrix, and a programmable register. The logic array provides five product terms per macrocell for implementing logic functions. The product-term select matrix allocates these terms either as primary logic inputs to OR and XOR gates, or as secondary inputs to the register's preset, clock, and clock enable controls.
- **Expander product terms**: When complex logic functions require more than the five product terms available per macrocell, the device provides expander product terms. Shareable expanders give each LAB 16 uncommitted single product terms — one from each macrocell — with inverted outputs that feed back into the logic array. Parallel expanders are unused product terms from one macrocell that can be borrowed by a neighbouring macrocell to implement fast, complex logic functions.
- **Programmable Interconnect Array (PIA)**: The PIA is a global bus that connects any signal source to any destination on the device. All dedicated inputs, I/O pins, and macrocell outputs feed into the PIA, making signals available throughout the entire device. Only the signals required by each LAB are actually routed from the PIA into that LAB — ensuring efficient use of routing resources.
- **I/O Control Blocks**: The I/O control block allows each I/O pin to be individually configured for input, output, or bidirectional operation. All I/O pins have a tristate buffer controlled by one of the global output enable signals or directly connected to ground or VCC. When connected to ground the pin is tri-stated as a dedicated input; when connected to VCC the output is actively driven.

6.2.3 In-System Programmability (ISP)

The MAX 7000AE supports In-System Programmability — the device internally generates the voltages needed to write its own EEPROM cells, allowing reprogramming directly on the board through the JTAG interface using only the standard 3.3V supply. During programming, all I/O pins are tri-stated and pulled up to 50 kΩ to prevent board conflicts.

The device also implements an ISP_Done bit — the last bit programmed — which keeps all I/O pins in a safe tri-stated state until programming is fully complete. In practice, ISP made the development cycle smooth — modify the VHDL, recompile in Quartus II, and re-download to the CPLD in one step without physically touching the test board.

6.3 Pin Out Diagram of CPLD

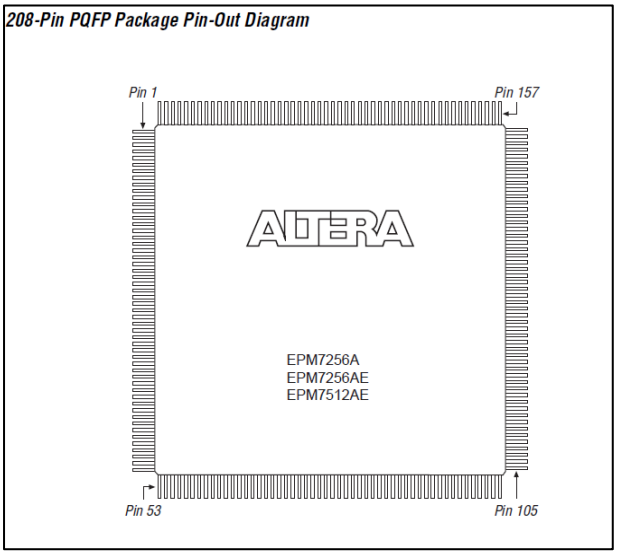


Figure 4: Pin Out diagram of CPLD

7. PROJECT WORK

7.1 Project Objective

In this project a Pseudo random code of 10 bits is generated on Altera-Max EPM7256E, CPLD. Out of the 10-bit Pseudo random pattern, we are tapping in 4-random bits are tapped, which is sent over the LVDS transmitter to LVDS receiver. The receiver output is given to an error detection module developed the CPLD, which compares the both transmitter input and receiver output and correspondingly generates an error if any mismatch between the signals is present.

Hardware and software used for this:

CPLD	Altera MAX EPM7256AEQC208-7
Test devices	LVDS transmitter, LVDS receiver
Clock source	24 MHz crystal oscillator
Simulation tool	ModelSim-Altera 10.1d
Development tool	Quartus II 13.1
Language	VHDL

7.2 Block-Level View of the Design

The system can be understood at a block level before going into the VHDL details. A master clock, generated from the 24 MHz crystal oscillator, is fed into a clock divider, which produces the working clock for the rest of the circuit. This clock drives the pseudo-random pattern generator, whose output is sent as the Tx input to the LVDS transmitter. The transmitter sends the data over a differential pair (D+/D−) to the LVDS receiver. The recovered Rx output is then compared against the original transmitted data by the comparator logic — if a mismatch occurs between the transmitted input data and the received output data, the comparator raises a bit error flag.

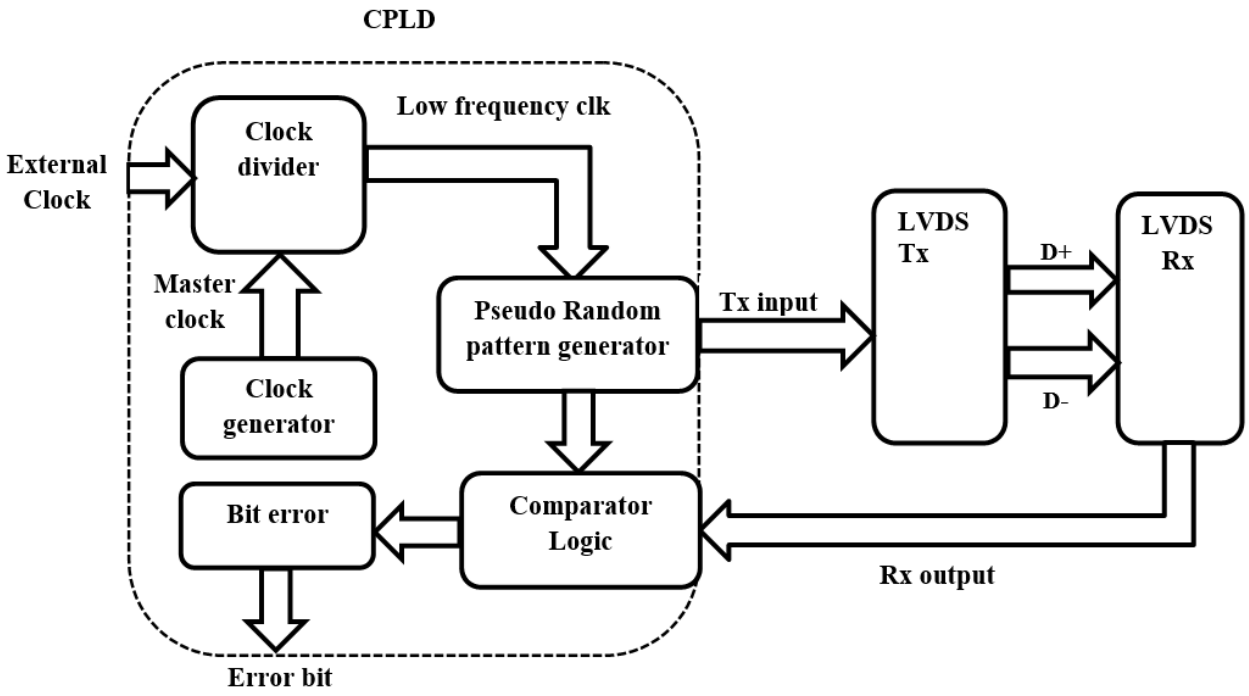


Figure 5: Block diagram of pseudo-random pattern generator using CPLD for LVDS Pair testing

7.3 Software Implementation

The design is implemented in VHDL and consists of four modules — clock generation, clock divider, pseudo-random pattern generator, and comparator with error detection.

7.3.1 Clock Generation Module

The clock generation module is for the generation of a clock through a 24 MHz crystal oscillator. The clock generation module consists of an inverter, resistor, and capacitor network for generating the clock frequency so as to obtain the required clock frequency. The crystal oscillator generates the clock, which is inverted and supplied to drive the circuit. The clock is forwarded to the buffer and then to the clock divider circuit.

A general clock generation circuit is shown below in the figure:

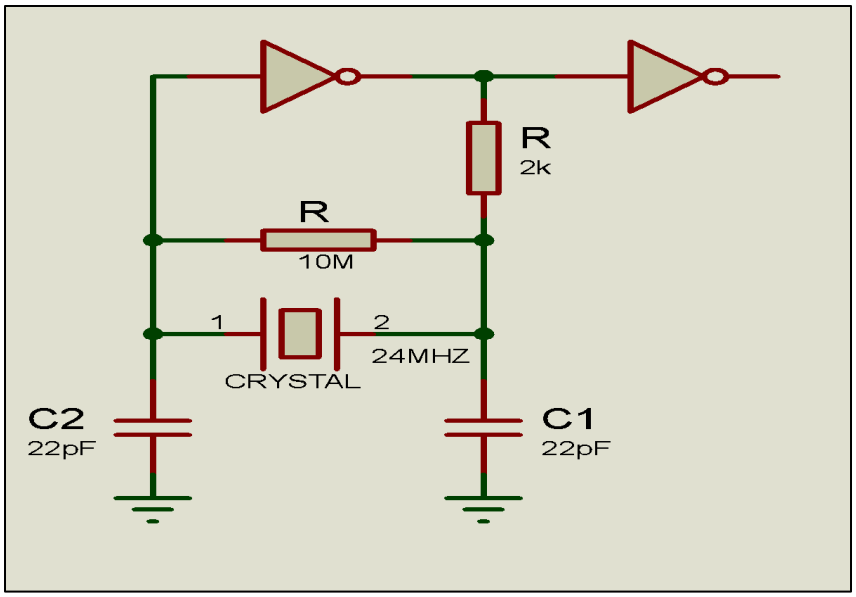


Figure 6: Clock generation circuit

7.4 Clock Divider and Selection

The clock generated by a crystal oscillator is of frequency 24Mhz. The 24Mhz clock is divided into multiple outputs of 12Mhz, 6 MHz, and 3Mhz, for the purpose of use by the random pattern. The random pattern is applied with different frequencies of clock pulses at different instances. The clock has been reduced to lower frequencies in order to test the system at different clock frequencies. The X and Y selection inputs are used to select the different clock frequencies to be input to the random counter. The table below shows different selection modes for the clock based on X and Y.

X	Y	Clock Selected
0	0	24 MHz
0	1	12 MHz
1	0	6 MHz
1	1	3 MHz

The clock generation and divider implementation was coded in VHDL and was implemented on the CPLD. In the code, the osc_ip corresponds to the clock generated from the crystal oscillator of frequency 24Mhz available on the CPLD. Two selection inputs are added in the code such that we can select a particular clock frequency that can be applied to the random

counter. The four outputs available at the output of a clock divider are selected using the two selection inputs.

7.5 The Pseudo-Random Pattern Generator Module

The pseudo-random pattern generator produces an output sequence based on its own previous state, using clock and reset as its two input pins. The module is implemented using a 10-bit shift register, where two of the bit outputs are combined using an XOR gate, and the result is fed back into the first bit of the register.

On reset, all 10 output bits are set to logic high (1). Once reset is released, the register begins shifting on every clock edge, with the XOR feedback continuously generating new bits, producing the pseudo-random sequence at the output.

Simulating this module confirms that all 10 output bits shift and toggle correctly according to the LFSR's feedback logic, producing the expected pseudo-random pattern.

7.6 Comparator and Error Detection Module

The comparator and error detection module compares the bit values of the pseudo-random output with the LVDS receiver's output and generates an error signal accordingly. The comparator takes the random pattern generated on the CPLD and the corresponding output from the LVDS receiver, and compares them bit by bit.

If any bit value does not match between the two, the module generates a logic high (1) error signal; if all bits match, the error signal remains at logic low (0). In this stage of the design, a 12 MHz clock is used to control the timing of the comparison.

7.7 Complete Logic and Code Implementation

For testing of the LVDS pair with pseudo-random data, logic and code are developed in VHDL. Here I am presenting the two different codes developed with the following difference:

a) **Logic with 24 MHz crystal clock and clock selection** — This version includes the clock generation and division circuitry, combined with the pattern generator and comparator logic to obtain the final objective of random implementation for different data rates. Two additional constant signals, referred to as G and G1 in the code, are included, since these are required as fixed inputs to the LVDS system. The signals count x, count y and count a define the number of clock pulse changes at which different frequencies of clock pulses are generated. The signals x and y are selection signals and are used in the selection of the clock pulse that is to be input to the random counter for generation of a random pattern. The output pin represents the

selection of different clock for different selection pins. This version was simulated and verified entirely in ModelSim.

```
libraryieee;
use ieee.std_logic_1164.all;
entity Kiran is
port(osc_in:instd_logic;
osc_out:inoutstd_logic;
rst:instd_logic;
D_in: inoutstd_logic_vector(9 downto 0);
R_out:inoutstd_logic_vector(9 downto 0);
output:inoutstd_logic;
error: outstd_logic);
end Kiran;

architecture behaviour of Kiran is
signal G:std_logic:='1';
signal G1:std_logic:='0';
signalx,y:std_logic;
signal clk_24:std_logic;
signal clk_12:std_logic:='0';
signalcount_x:integerrange 0 to 4;
signal clk_6:std_logic:= '0';
signalcount_y:integerrange 0 to 4;
signal clk_3:std_logic:='0';
signalcount_a:integerrange 0 to 4;
--clock divider for 24Mhz clock pulse--
begin
process(osc_in)
begin
osc_out<= not(osc_in);
endprocess;
process(osc_out)
begin
    clk_24<=osc_out;
endprocess;

--clock divider for 12Mhz clock pulse--
process(osc_out)
begin
if(count_x=1) then
count_x<=0;
```

```
    clk_12<= not (clk_12);
else
count_x<=count_x+1;
endif;
endprocess;

--clock divider for 6Mhz clock pulse---
process(osc_out)
begin
if(osc_out ='1')then
if (count_y= 1) then
count_y<= 0;
    clk_6<= not (clk_6);
else
count_y<= count_y+1;
endif;
endif;
endprocess;

--clock divider for 3 Mhz clock pulse--
process(osc_out)
begin
if(osc_out='1')then
if (count_a=3) then
count_a<=0;
    clk_3<=not (clk_3);
else
count_a<=count_a+1;
endif;
endif;
endprocess;

--selection--
process(clk_24)
begin
if(x='0'and y='0') then
    output<=clk_24;
elsif(x='0'and y='1') then
    output<=clk_12;
elsif(x='1'and y='0')  then
    output<=clk_6;
else
    output<=clk_3;
```

```
endif;
endprocess;

-----random------
process(output)
begin
if(output='1')then
if(rst='1') then
D_in(0)<='1'after 10 ps;
D_in(1)<='1'after 10 ps;
D_in(2)<='1'after 10 ps;
D_in(3)<='1'after 10 ps;
D_in(4)<='1'after 10 ps;
D_in(5)<='1'after 10 ps;
D_in(6)<='1'after 10 ps;
D_in(7)<='1'after 10 ps;
D_in(8)<='1'after 10 ps;
D_in(9)<='1'after 10 ps;

else
D_in(0)<=D_in(7) xorD_in(8) after 10 ps;
D_in(1)<=D_in(0)after 10 ps;
D_in(2)<=D_in(1)after 10 ps;
D_in(3)<=D_in(2)after 10 ps;
D_in(4)<=D_in(9) xorD_in(3) after 10 ps;
D_in(5)<=D_in(4)after 10 ps;
D_in(6)<=D_in(5)after 10 ps;
D_in(7)<=D_in(6)after 10 ps;
D_in(8)<=D_in(7)after 10 ps;
D_in(9)<=D_in(8)after 10 ps;
endif;
endif;
endprocess;

--comparison--
process(output)
begin
ifrising_edge(output) then
ifR_out(9 downto 0)=D_in(9 downto 0) then
error<='0';
else
error<='1';
endif;
```

```
endif;
endprocess;
end behaviour;
```

The Simulation results above logic with an external clock, clock selection and multiple clocks, have been shown below:

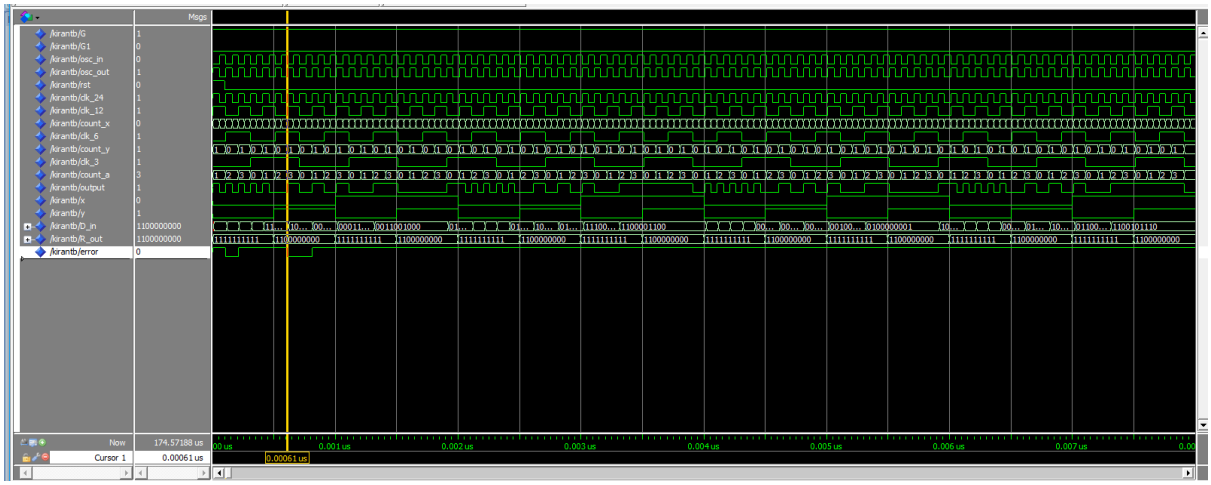


Figure 7: Simulation Wavefrom

b) **Logic with direct external clock** — This version removes the internal clock generation and division logic, and takes the clock directly as an input instead. This is the version that is carried forward into Quartus II for synthesis and implementation on the actual CPLD hardware.

```
libraryieee;
use ieee.std_logic_1164.all;
entity cpld_kiran is
port(clk:instd_logic;
rst:instd_logic;
       clk_1:inoutstd_logic:='1';
       clk_2:inoutstd_logic:='0';
D_in: inoutstd_logic_vector(9 downto 0);
R_out:inoutstd_logic_vector(9 downto 0);
       G: outstd_logic:='1';
       G1: outstd_logic:='0';
error: inoutstd_logic_vector(0 downto 0));
end cpld_kiran;
```

```
architecture behaviour of cpld_kiran is

--clock divider for 24Mhz clock pulse--
--clock 1 running on rising edge of clock---
begin
process(clk)
begin
ifrising_edge (clk) then
clk_1<=not(clk_1);
endif;
endprocess;
---clock 2 running on faling edge of clock -------
process(clk)
begin
iffalling_edge (clk) then
clk_2<=not(clk_2);
endif;
endprocess;

-----random------
process(clk_1)
begin
ifrising_edge(clk_1) then
if(rst='1') then
D_in(0)<='1';
D_in(1)<='1';
D_in(2)<='1';
D_in(3)<='1';
D_in(4)<='1';
D_in(5)<='1';
D_in(6)<='1';
D_in(7)<='1';
D_in(8)<='1';
D_in(9)<='1';
else
D_in(0)<=D_in(7) xorD_in(8) ;
D_in(1)<=D_in(0);
D_in(2)<=D_in(1);
D_in(3)<=D_in(2);
D_in(4)<=D_in(9) xorD_in(3);
D_in(5)<=D_in(4);
D_in(6)<=D_in(5);
D_in(7)<=D_in(6);
```

```
D_in(8)<=D_in(7);
D_in(9)<=D_in(8);
endif;
endif;
endprocess;

--comparison--
process(clk_2)
begin

iffalling_edge(clk_2) then
ifR_out(0)=D_in(0) and
R_out(3)=D_in(3) and
R_out(6)=D_in(6) and
R_out(9)=D_in(9) then

error(0)<='0';
else
error(0)<='1';
endif;
endif;
endprocess;
end behaviour;
```

7.8 Synthesis Report

Following simulation, the direct-external-clock version of the design was taken into Quartus II for synthesis and implementation on the CPLD:

- **RTL view** — Quartus II's RTL viewer was used to view the implemented logic code translated into its constituent logic blocks and interconnections, providing a useful check on how the VHDL code had actually been synthesised, including the input and output connections.

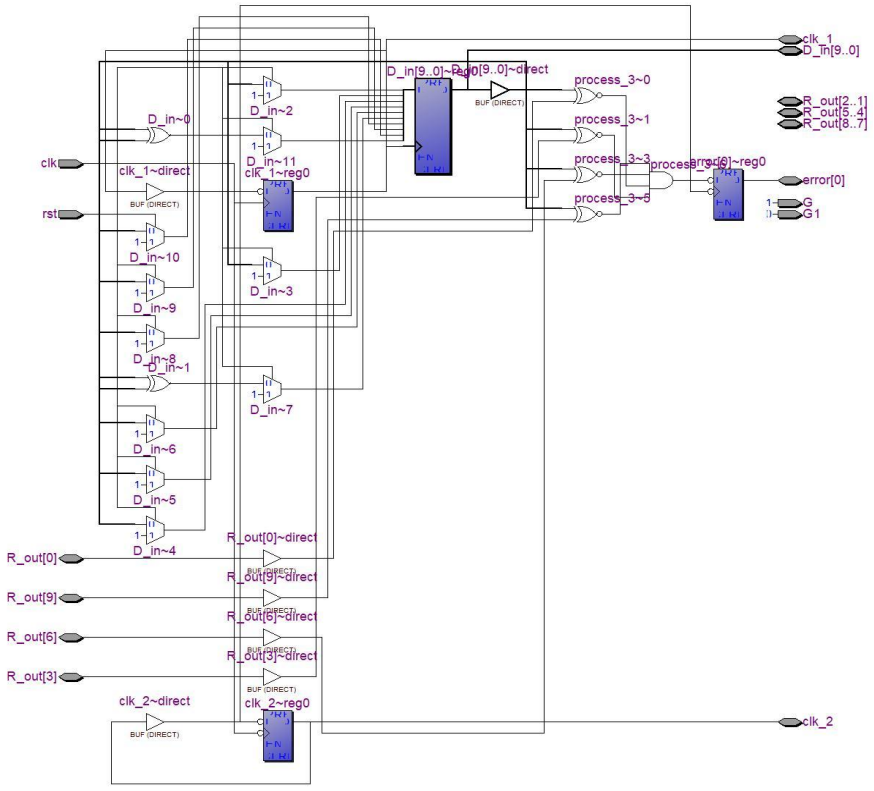


Figure 8: RTL view of the implemented logic

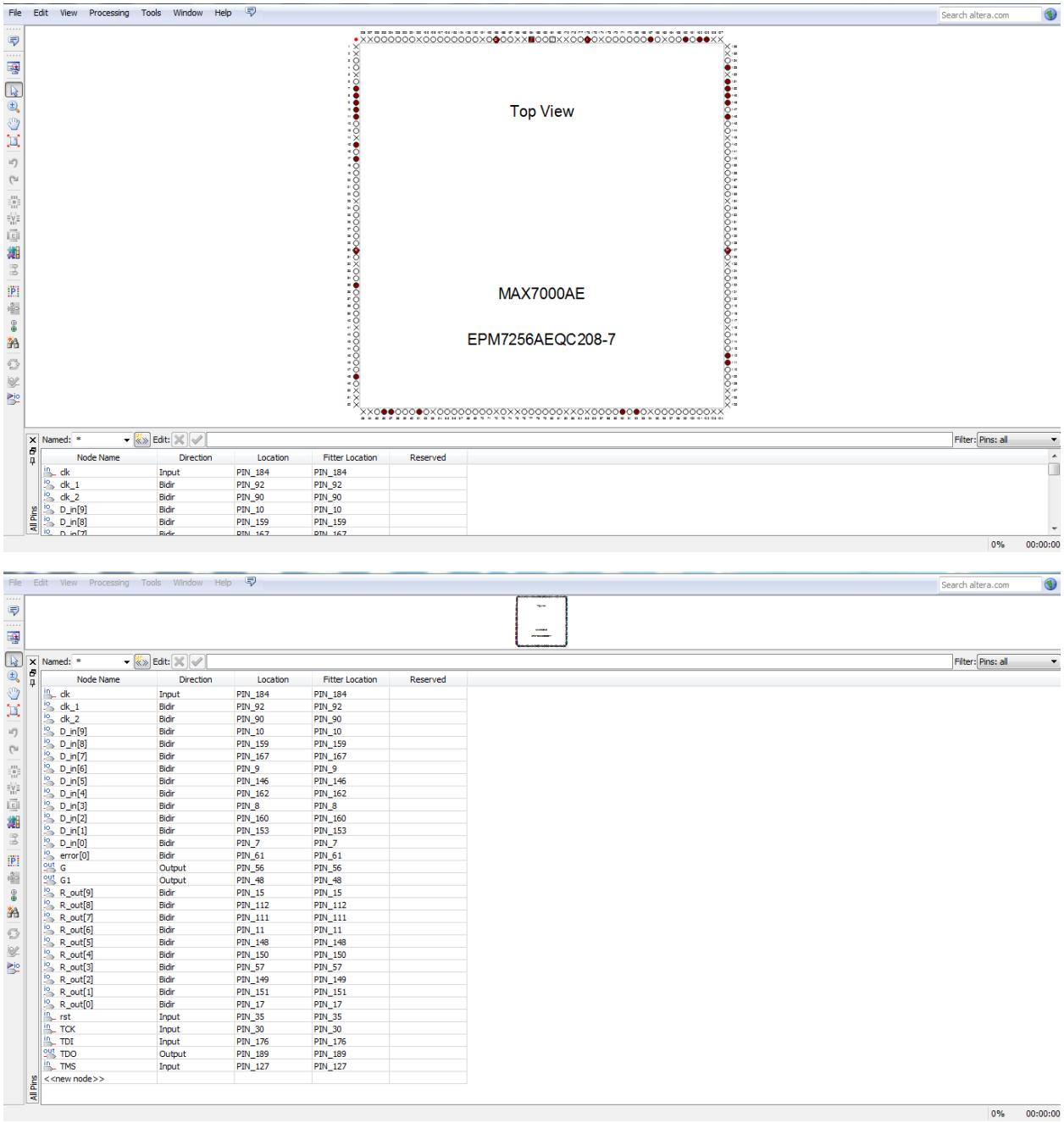
- **Synthesis flow summary** — The synthesis flow summary reports the status of the design, the target device, and the resources used. For this design, the flow summary reported the target device as the EPM7256AEQC208-7 (MAX7000AE family), with 26 macrocells used out of 256 available (10%), and 31 pins used out of 164 available (19%).

Flow Summary	
Flow Status	Successful - Tue Jun 30 12:37:31 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	lvds
Top-level Entity Name	lvds
Family	MAX7000AE
Device	EPM7256AEQC208-7
Timing Models	Final
Total macrocells	26 / 256 (10 %)
Total pins	31 / 164 (19 %)

Figure 9: Simulation and Synthesis Flow summary

- **Pin assignment** — Each input and output pin in the design — clock, data inputs/outputs, error output, and so on — was assigned to a specific physical pin on

the CPLD using the Pin Planner in Quartus II, with the pin diagram and datasheet referred to carefully to confirm correct voltage and current levels before assignment.



Node Name	Direction	Location	Fitter Location	Reserved
clk	Input	PIN_184	PIN_184	
clk_1	Bidir	PIN_92	PIN_92	
clk_2	Bidir	PIN_90	PIN_90	
D_in[9]	Bidir	PIN_10	PIN_10	
D_in[8]	Bidir	PIN_159	PIN_159	
D_in[7]	Bidir	PIN_167	PIN_167	
D_in[6]	Bidir	PIN_9	PIN_9	
D_in[5]	Bidir	PIN_146	PIN_146	
D_in[4]	Bidir	PIN_162	PIN_162	
D_in[3]	Bidir	PIN_8	PIN_8	
D_in[2]	Bidir	PIN_160	PIN_160	
D_in[1]	Bidir	PIN_153	PIN_153	
D_in[0]	Bidir	PIN_7	PIN_7	
error[0]	Bidir	PIN_61	PIN_61	
G	Output	PIN_56	PIN_56	
G1	Output	PIN_48	PIN_48	
R_out[9]	Bidir	PIN_15	PIN_15	
R_out[8]	Bidir	PIN_112	PIN_112	
R_out[7]	Bidir	PIN_111	PIN_111	
R_out[6]	Bidir	PIN_11	PIN_11	
R_out[5]	Bidir	PIN_148	PIN_148	
R_out[4]	Bidir	PIN_150	PIN_150	
R_out[3]	Bidir	PIN_57	PIN_57	
R_out[2]	Bidir	PIN_149	PIN_149	
R_out[1]	Bidir	PIN_151	PIN_151	
R_out[0]	Bidir	PIN_17	PIN_17	
rst	Input	PIN_35	PIN_35	
TCK	Input	PIN_30	PIN_30	
TDI	Input	PIN_176	PIN_176	
TDO	Output	PIN_189	PIN_189	
TMS	Input	PIN_127	PIN_127	
<<new node>>				

Figure 10: Pin assignment of EPM7256AEQC208-7

- **Fitter placement and verification** — After the Fitter tool verified the assigned pins, the pin assignment could be viewed as a top view of the CPLD package. Of the 31 pins programmed in total, 19 pins were the functionally significant pins actually utilised

and targeted by the design, with the remaining pins corresponding to supply, ground, and other supporting connections assigned automatically.

- **Code download onto the CPLD** — Once the Fitter step confirmed valid pin placement, the compiled design was downloaded onto the CPLD using Quartus II's Programmer in JTAG mode. The Programmer showed the target device (EPM7256AEQC208-7) with Program/Configure and Verify selected, and the download completed with a progress reading of 100% successful, confirming the design was correctly programmed and ready for hardware testing.

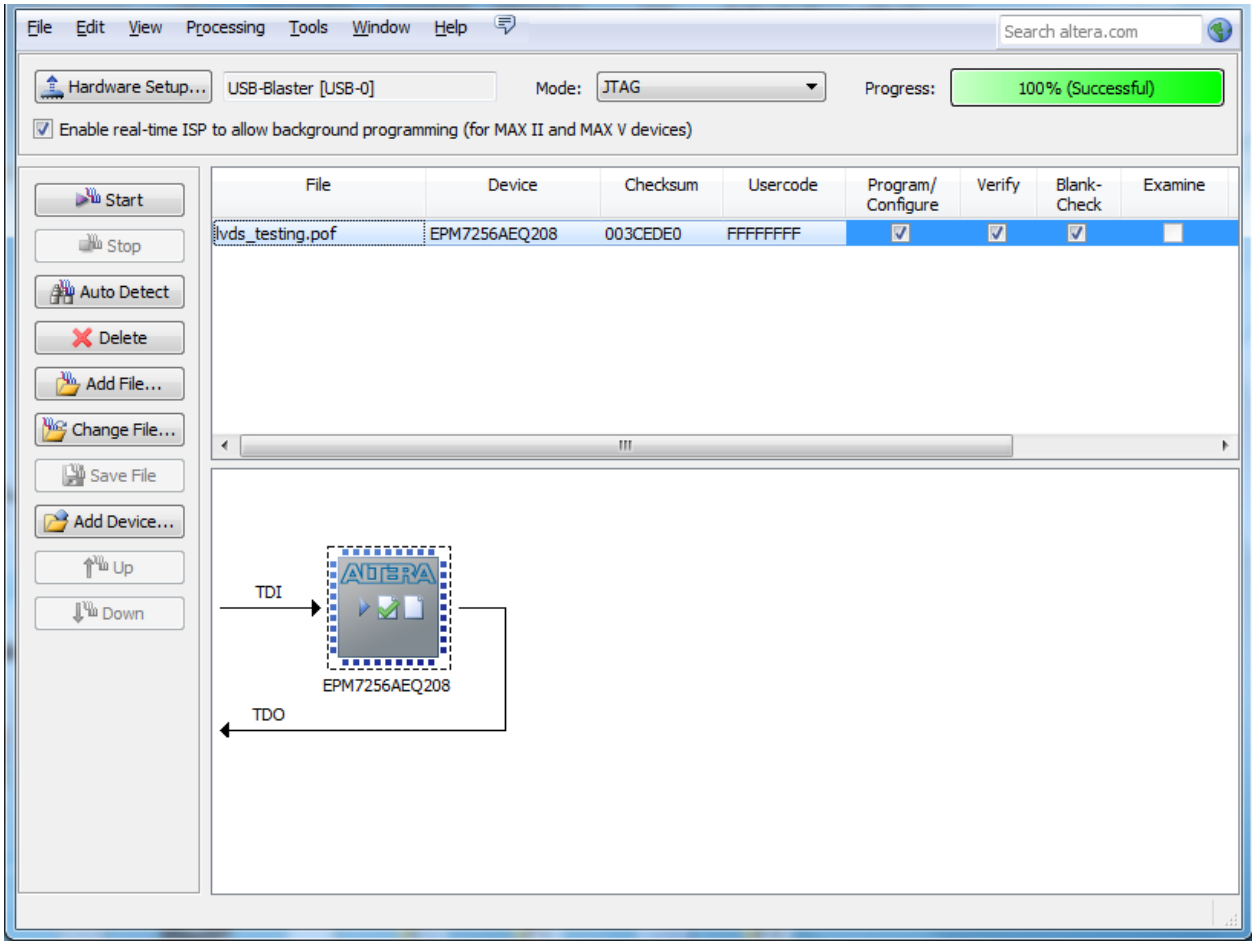


Figure 11: Program Download

8. HARDWARE IMPLEMENTATION AND TEST RESULTS

This covers the hardware setup used for testing, along with the results obtained from the oscilloscope. The oscilloscope was used to capture data at a single fixed frequency of 12 MHz, showing all four data input bits to the LVDS transmitter separately, along with the corresponding four receiver output bits and the resulting comparison (error) output.

8.1 Hardware Setup

With the design successfully downloaded onto the CPLD, testing moved to the physical test board. The board carries the programmed CPLD, the LVDS transmitter and receiver, connected to a power supply, an oscilloscope for signal capture and analysis.

Figure 12: LVDS Test Board

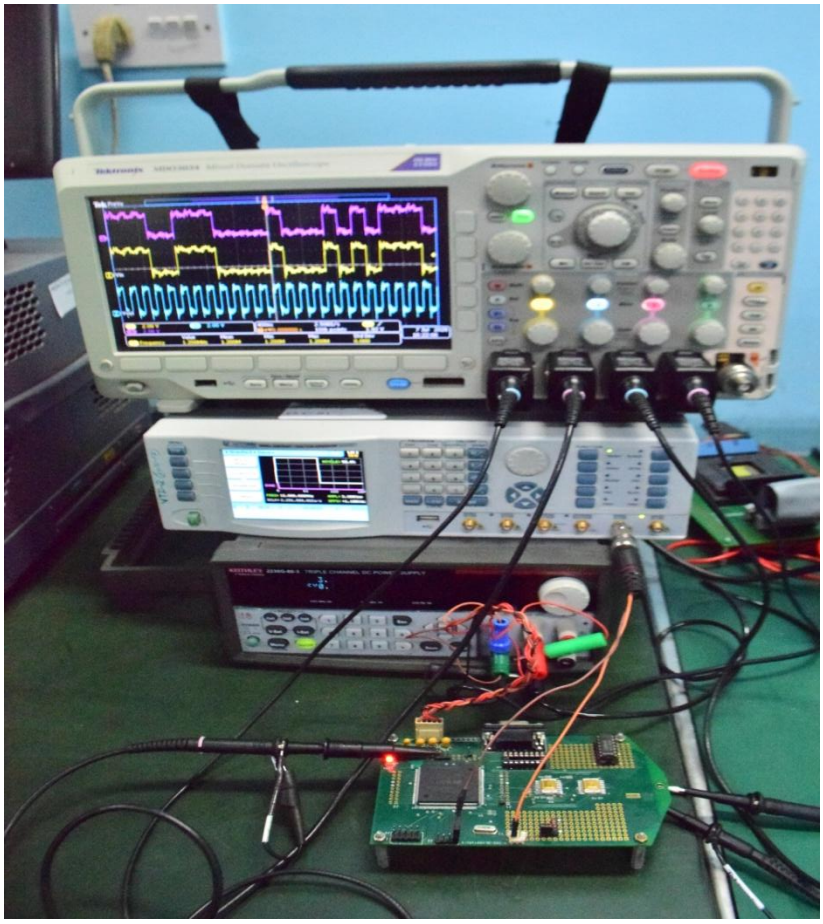


Figure 13: LVDS Test Setup

8.2 Test Conditions

Device	Vdd (V)	Idd (mA), static
CPLD	3.3	40.0
LVDS Tx	3.3	19.3
LVDS Rx	3.3	30.1

8.3 Oscilloscope Observations

The random data generated by the CPLD is being transmitted to the LVDS transmitter. In the wave forms shown below the first waveform corresponds to the data of the receiver in pink colour, the random pattern from a single lead is shown in blue colour, corresponding clock shown lead in light blue colour.

The oscilloscope outputs of one data bit are shown below.

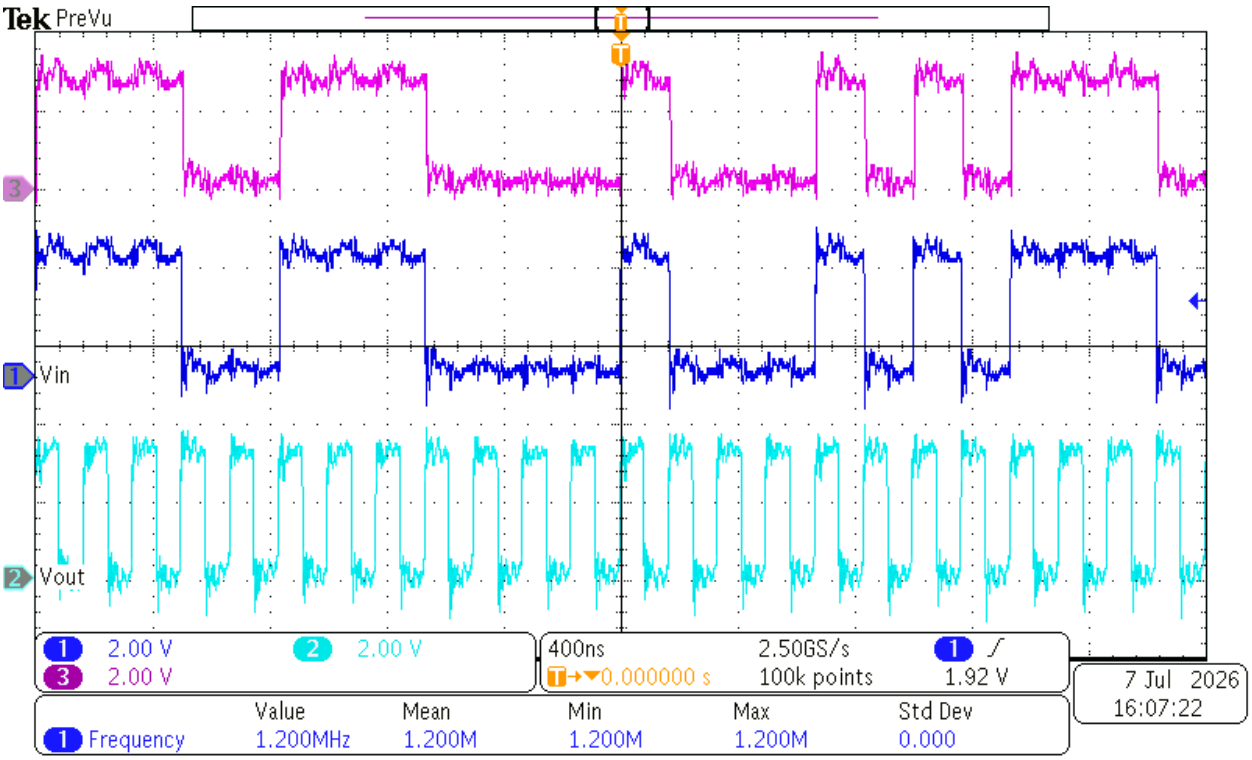


Figure 14: Oscilloscope Result

The waveform above shows that the transmitted data in blue color and the received data in pink color are matching at **1.2 MHz frequency**, confirming **zero error** at this frequency. Similarly, the same results were observed for the remaining three data bits.

8.4 Test Results and Analysis

This section analyses the test results obtained, along with the propagation delays measured at different operating frequencies. When transmitted and received data mismatch within an LVDS pair, an individual error bit is generated for that pair. Since there are four LVDS pairs in this design, the overall error bit for the system is obtained by an AND operation of these four individual error bits — meaning the overall error is flagged only when at least one pair shows a mismatch, and stays at zero only when all four pairs match correctly. Based on this, the measured delays have been grouped into two sets: one for the condition where no error is observed, and another for the condition where an error bit is observed.

9. CONCLUSION AND FUTURE WORK

9.1 Conclusion

This project covered the design, simulation, and hardware implementation of a pseudo random pattern generator on a CPLD using VHDL coding on Modelsim, and testing the LVDS transmitter-receiver system using this data pattern. The clock generation, 10-bit LFSR-based pattern generator, and comparator modules were each verified individually in simulation before integration, and the complete design was synthesised and downloaded onto the Altera MAX EPM7256AEQC208-7 CPLD.

Testing using an oscilloscope confirmed error-free transmission up to approximately 40 MHz, with errors appearing at 45 MHz and above. This confirmed that a design which simulates correctly still needs to be verified on actual hardware, since real timing margins and signal integrity affect behaviour in ways that simulation alone does not capture.

9.2Future Work

The following directions are suggested for extending this work further:

- Extending the pattern generator to a longer LFSR sequence to test the LVDS pair under a wider range of bit patterns and transition densities.
- Implementing the design on an FPGA to explore higher clock speeds and more complex pattern generation algorithms.